

1. Introduction

1.1 Overview of the Project

The project focuses on the development of a **Business Management System** designed to streamline and integrate core business processes such as sales, purchases, inventory management, finance, and human resources. The system is implemented as a database-driven application, ensuring that all modules are connected to a centralized database for real-time data access and consistency. By consolidating different operations into one platform, this system improves efficiency, reduces redundancy, and provides accurate insights for decision-making.

1.2 Problem Statement

In many small- to medium-scale manufacturing and trading businesses, operations are often managed through fragmented systems or even manual record-keeping. This leads to issues such as:

- Data duplication and inconsistency across different departments.
- Lack of real-time updates, resulting in delays in decision-making.
- Difficulty in generating accurate financial reports and forecasts.
- Security risks due to lack of controlled access.

These limitations reduce productivity and increase operational costs. Hence, there is a need for a centralized system with a strong database backbone that ensures data integrity, scalability, and security.

1.3 Importance of the System

The Business Management System provides the following benefits:

- **Data Centralization:** Integrates all business modules into a single database, eliminating redundancy.
- **Improved Decision-Making:** Real-time access to data supports accurate reporting and forecasting.
- **Operational Efficiency:** Automates processes like sales, purchases, and financial tracking.
- **Scalability:** The database design allows the system to adapt as the business grows.
- **Security and Control:** Role-based access ensures that sensitive operations are only available to authorized users.

2. Objectives

The primary objective of this project is to design and implement a database-driven Business Management System that integrates core business functions into a unified platform. The system aims to provide efficient data management, support business operations, and improve decision-making.

The specific goals are as follows:

- To design a normalized and efficient relational database schema for modules such as **Sales, Purchases, Inventory, Customers, Employees, and Finance**.
- To ensure **data accuracy, integrity, and consistency** through constraints, relationships, and transaction management.
- To provide **role-based access control** so that administrators, staff, and managers can perform only their assigned tasks.
- To implement **CRUD operations** (Create, Read, Update, Delete) for all entities, ensuring usability and flexibility.
- To enable **report generation and forecasting** through optimized SQL queries and stored procedures.
- To reduce redundancy and improve performance by applying **normalization** and indexing techniques.
- To support scalability and adaptability so the system can handle increasing business data over time.
- To improve overall operational efficiency by **automating financial tracking, inventory updates, and customer management**.

3. Requirements Specifications

The Business Management System requires both **functional** and **non-functional** requirements to ensure it meets the needs of end-users while maintaining efficiency, reliability, and security. Additionally, user roles are defined to establish access privileges.

3.1 Functional Requirements

The functional requirements describe the core operations the system must support. These are primarily **CRUD operations** along with reporting and validation features.

- **User Management** – Add, edit, delete, and search users with authentication and role assignment.
- **Customer Management** – Manage customer records (add, update, search, and delete).
- **Supplier Management** – Handle supplier information including contact details and supply history.
- **Product & Inventory Management** – Maintain product catalog, stock updates, and category management.
- **Sales Management** – Create and update sales transactions, generate invoices, and track payment status.
- **Purchase Management** – Record and manage purchases including supplier details and material units.
- **Finance Management** – Record incomes and expenses, generate budgets, and compare with actual financial results.
- **Report Generation** – Generate reports on sales, purchases, products, customers, income, and expenses with export options.

3.2 Non-Functional Requirements

Non-functional requirements define system qualities that ensure usability and performance:

- **Performance:** The system must respond to queries (e.g., search, save, edit) within 2 seconds for 95% of requests.
- **Scalability:** The database must support growth up to 500+ concurrent users.
- **Security:** All users must be authenticated, and sensitive data must be restricted through role-based access control.
- **Data Integrity:** Transactions must follow ACID properties to prevent data loss or inconsistency.
- **Usability:** The interface should be intuitive, with common tasks (e.g., adding sales or generating reports) completed in under 3 steps.
- **Reliability:** The system must handle failures gracefully with rollback and recovery mechanisms.

3.3 User Roles

Different users interact with the system, each with specific privileges:

- **Admin**
 - Manage system users (create, edit, delete accounts).
 - Configure system settings and assign roles.
 - Access all modules and reports.
- **Manager**
 - Oversee purchases, sales, and financial operations.
 - Generate reports and forecasts.
 - Manage employees and suppliers.
- **Staff**
 - Record day-to-day sales and purchases.
 - Update product and customer details.
 - Limited access to reporting functions.
- **Accountant**
 - Manage income and expense records.
 - Generate financial reports.
 - Assist in budget and forecasting tasks.

This role-based structure ensures both **security** and **operational efficiency**, as each user only has access to the functionalities relevant to their responsibilities.

4. System Design

4.1 System Architecture

The system adopts a **Three-Tier Architecture**, which is fundamental for separating data management responsibilities and optimizing the database's role.

- **Presentation Tier (Client):** The user interface (Web-based) for inputting and displaying data.
- **Application Tier (Business Logic/Service Layer):** Contains the core business rules and, critically, the **Data Access Layer (DAL)**. This tier is the only component allowed to directly communicate with the Database Tier, ensuring all data operations are validated and controlled.
- **Database Tier (RDBMS):** Houses the relational database (e.g., MySQL, PostgreSQL). Its primary roles are **data storage, integrity enforcement** (via constraints, indexing, and triggers), and **transaction management**.

The use of the **Repository Pattern** in the Application Tier acts as an abstraction layer, shielding the business logic from specific database SQL details, which is crucial for advanced database maintainability and portability.

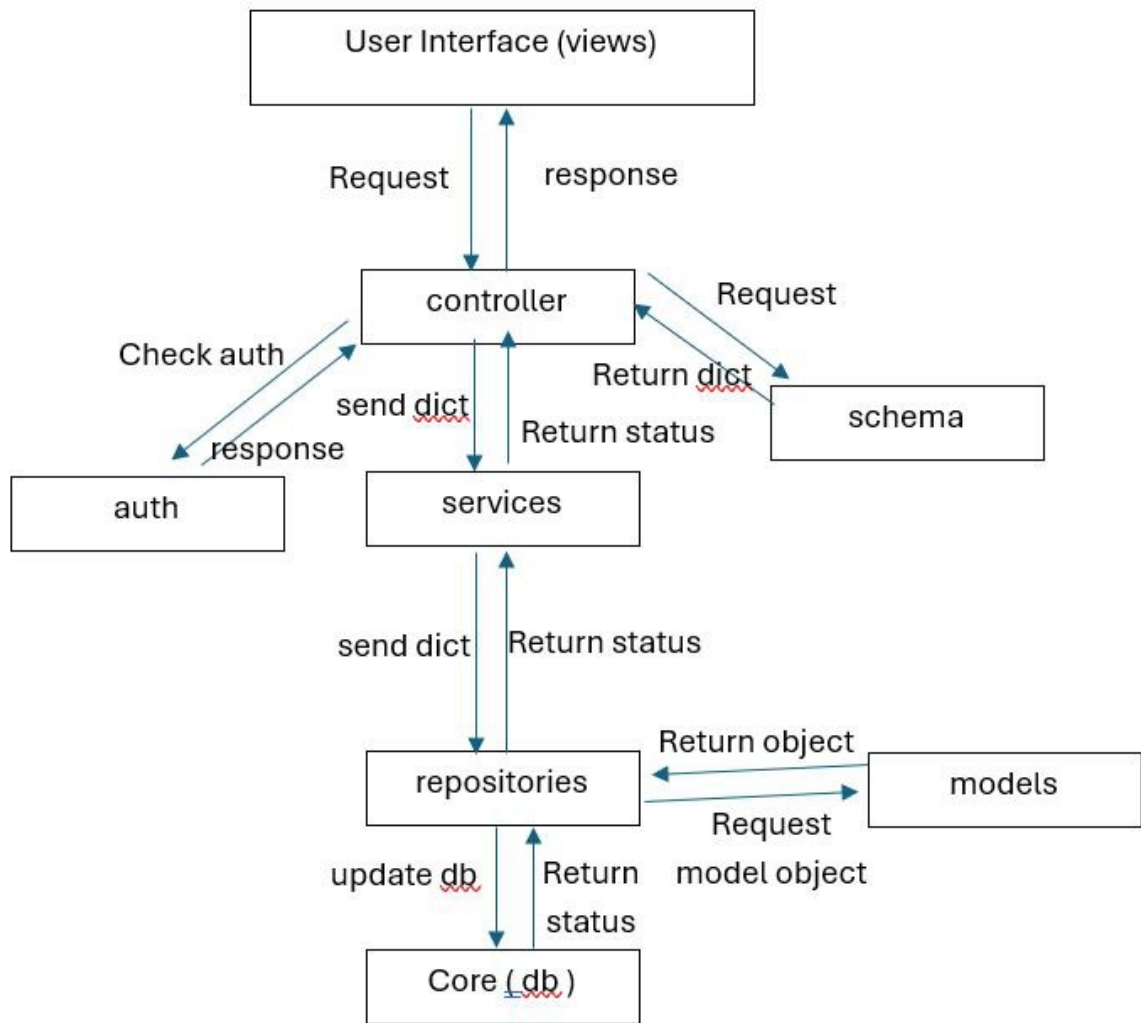


Fig. 4.1.1 System Architecture Design

4.2 Use Case Diagrams

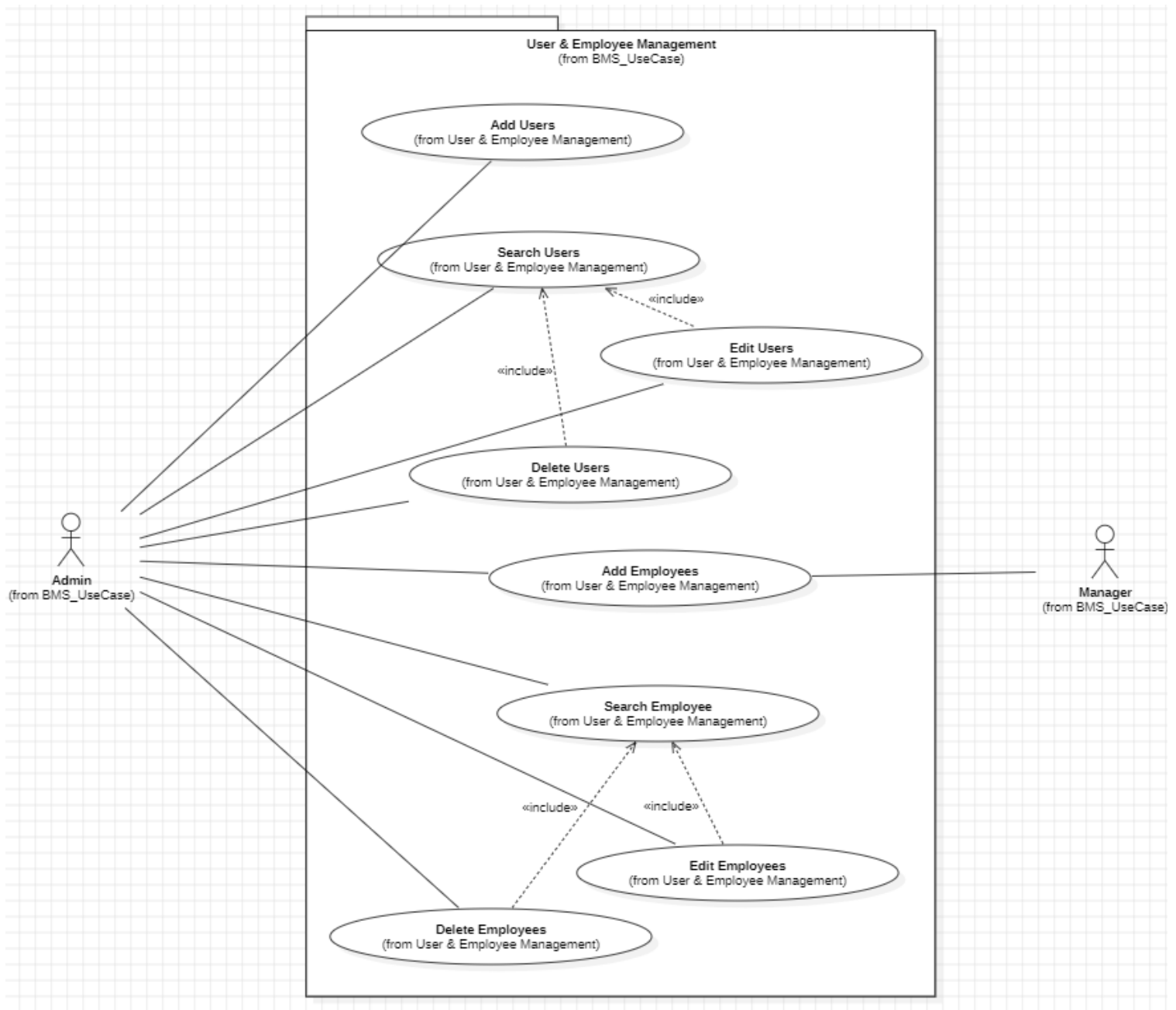


Fig. 4.2.1 Use Case Diagram for User and Employee Management

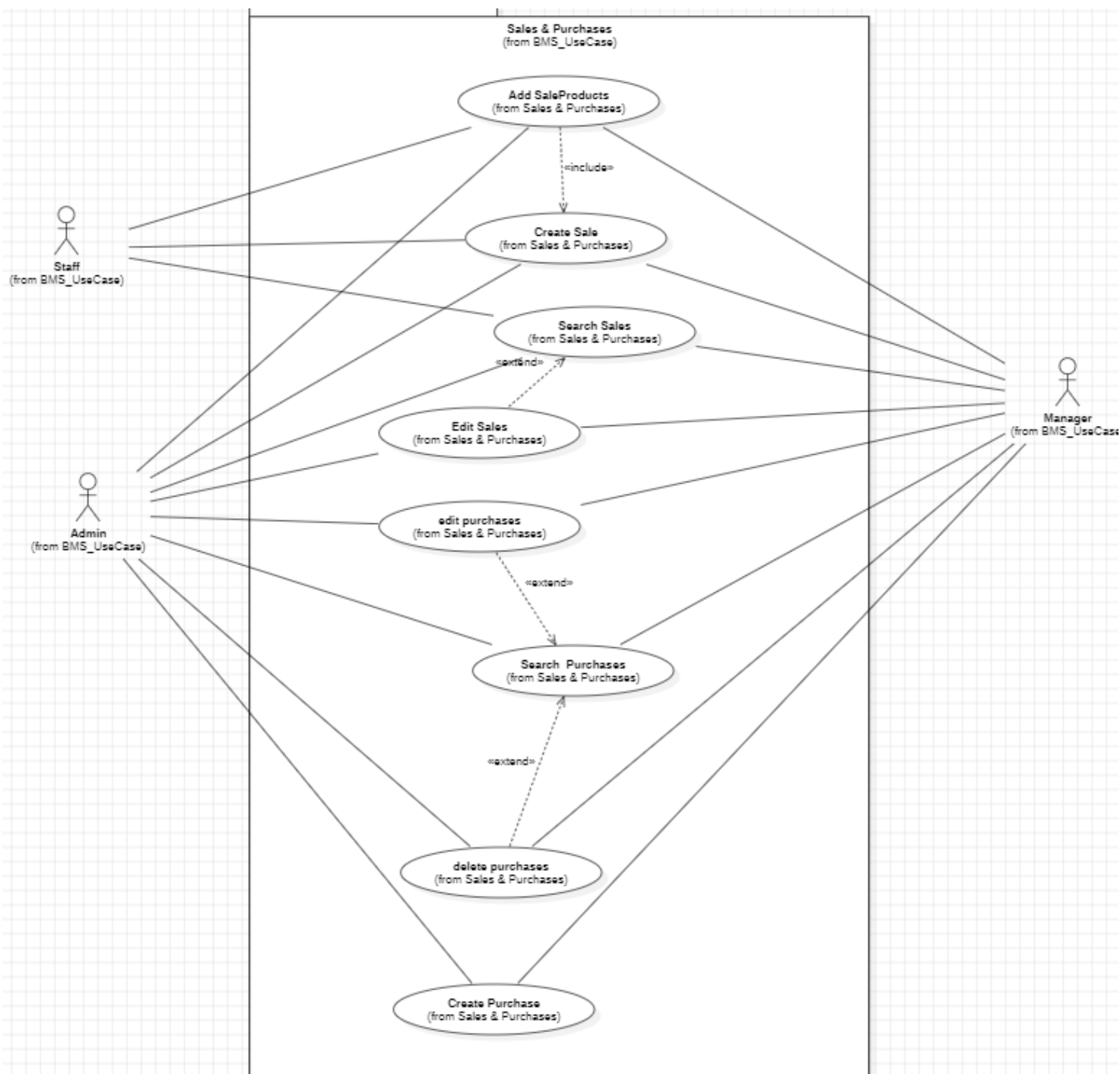


Fig. 4.2.2 Use Case Diagram for Sales and Purchases

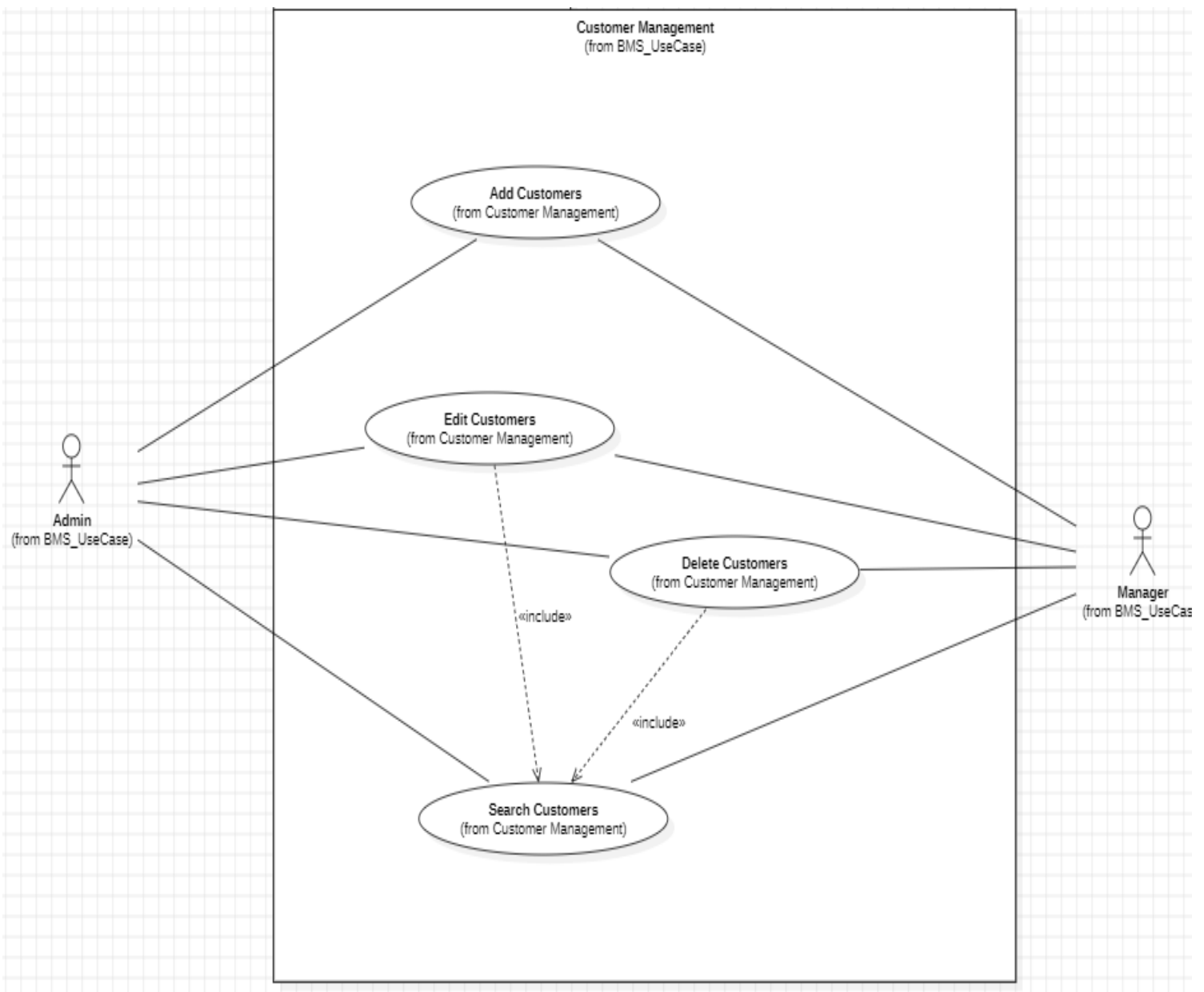


Fig. 4.2.3 Use Case Diagram for Customer Management

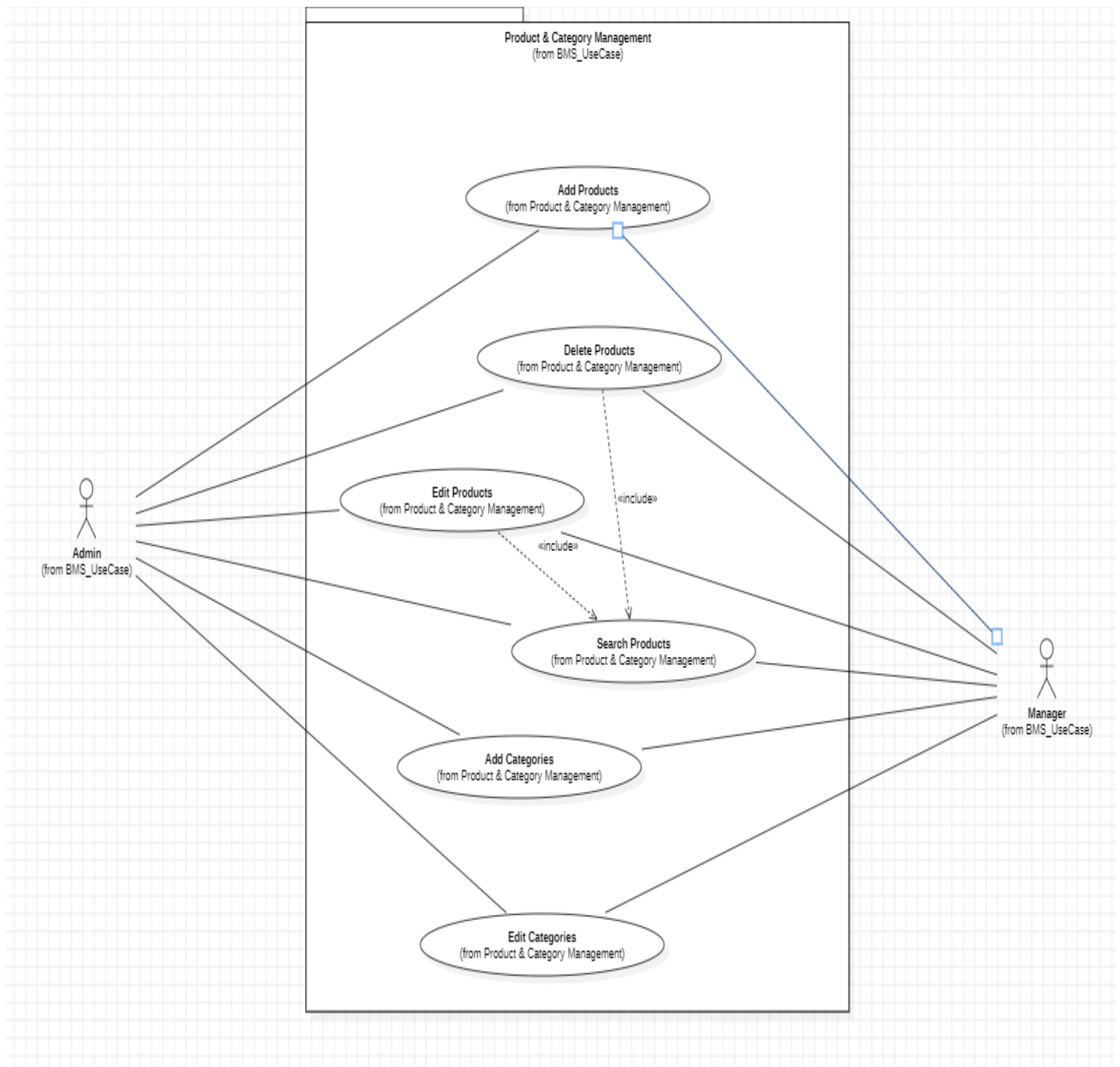


Fig. 4.2.4 Use Case Diagram for Product and Category Management

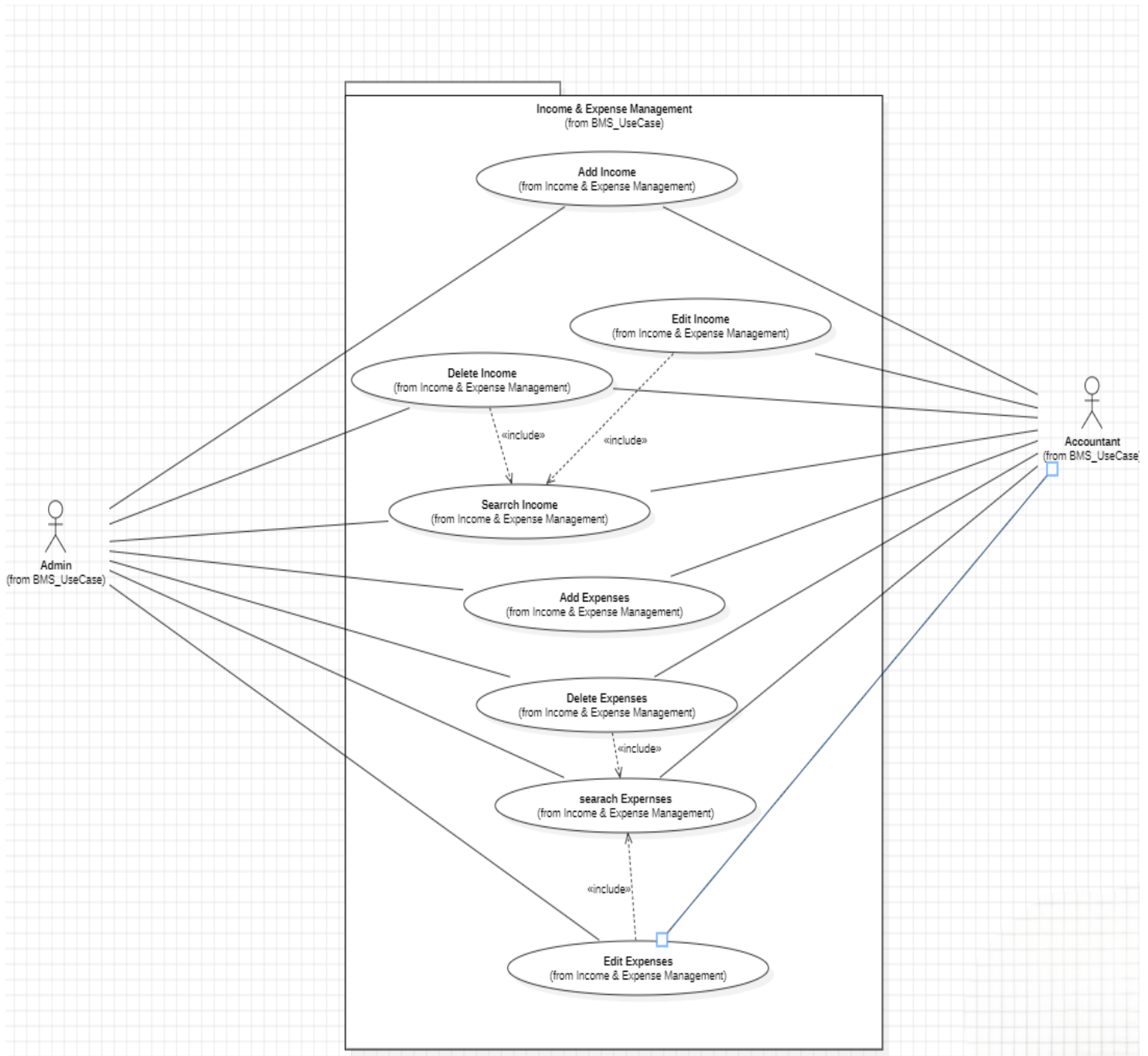


Fig. 4.2.5 Use Case Diagram for Income and Expense Management

4.3 System Flow Chart

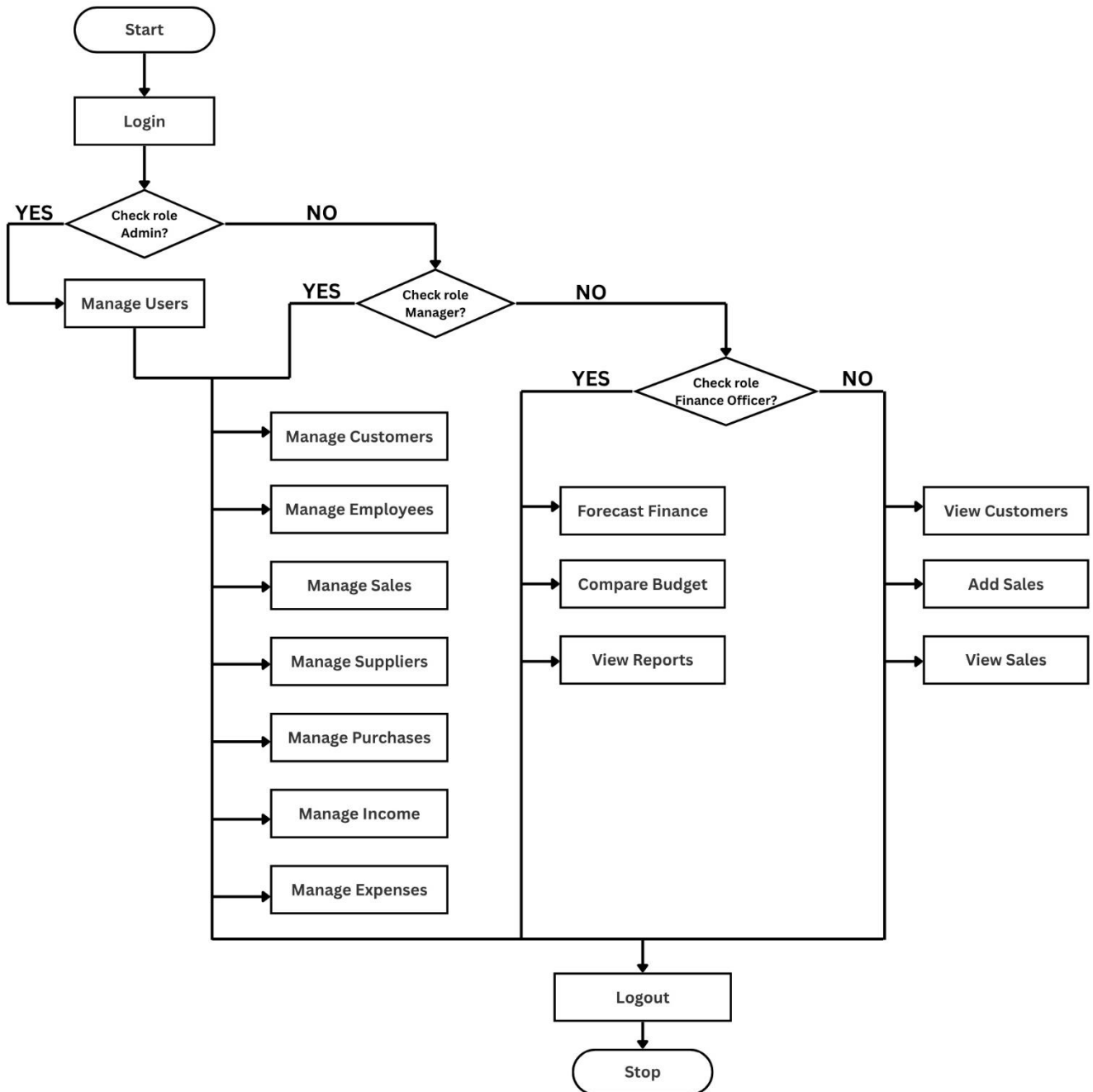


Fig. 4.3.1 System Flow Chart

5. Database Design

5.1 ER Diagram

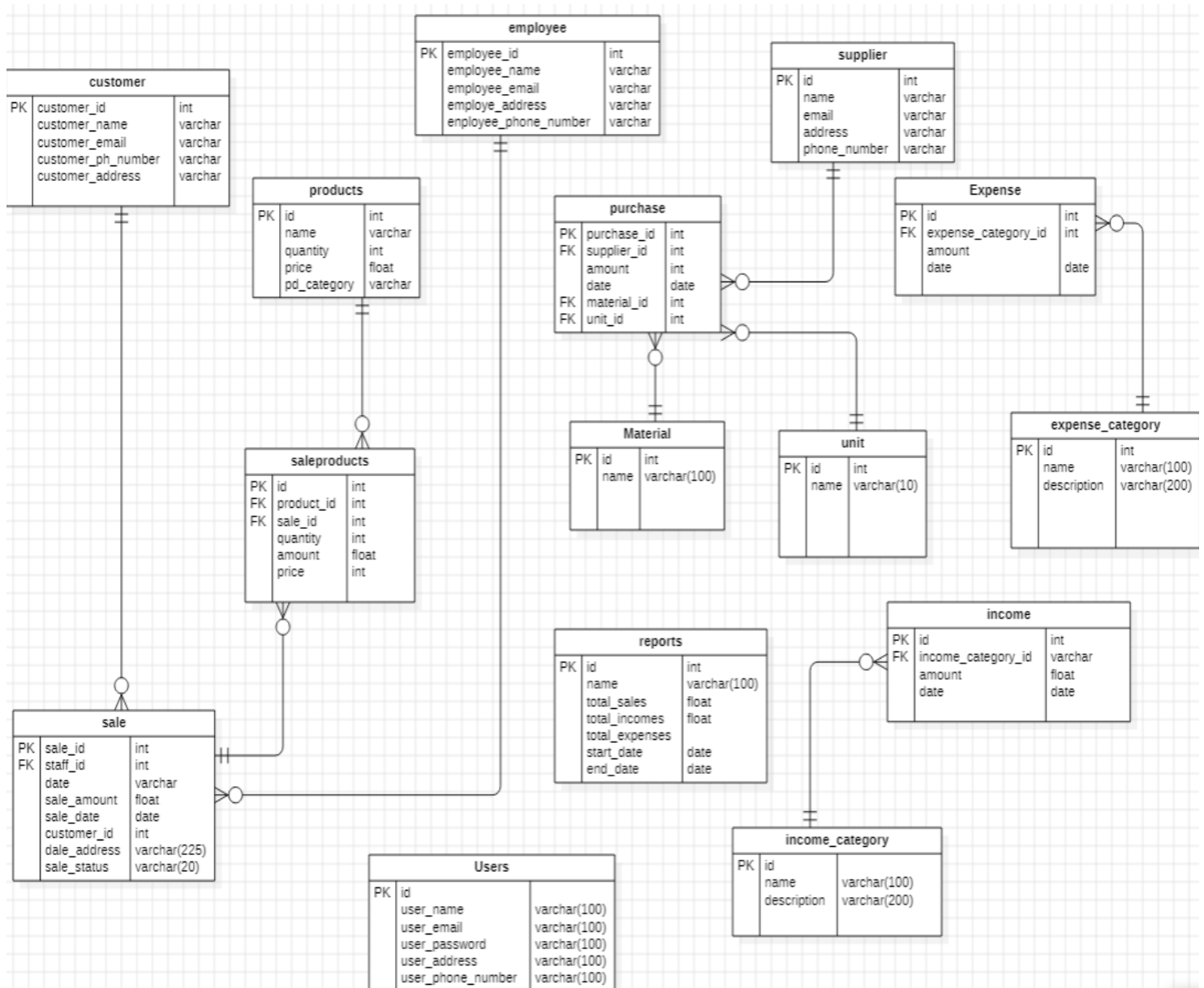


Fig. ER Diagram

5.2 Relational Schema

Table 5.2.1 Relational Schema for Customers

Table	Attributes	Type	Constraints	PK/FK	FK References
Customers	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	customer_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	customer_name	VARCHAR(50)	NOT NULL	—	—
	customer_email	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	customer_password	VARCHAR(50)	NOT NULL	—	—
	customer_phone_number	VARCHAR(50)	NOT NULL	—	—
	customer_address	VARCHAR(50)	NOT NULL	—	—

Table 5.2.2 Relational Schema for Products

Table	Attributes	Type	Constraints	PK/FK	FK References
Products	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	product_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	product_name	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	product_category	VARCHAR(50)	NOT NULL	—	—
	product_price	VARCHAR(50)	NOT NULL	—	—
	product_quantity	VARCHAR(50)	NOT NULL	—	—

Table 5.2.3 Relational Schema for Income

Table	Attributes	Type	Constraints	PK/FK	FK References
Incomes	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	income_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	income_category	VARCHAR(50)	NOT NULL	—	—
	amount	FLOAT	NOT NULL	—	—
	description	VARCHAR(200)	NOT NULL	—	—
	date	DATE	NOT NULL	—	—

Table 5.2.4 Relational Schema for Income_Categories

Table	Attributes	Type	Constraints	PK/FK	FK References
Income_Categories	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
Income_Categories	name	VARCHAR(100)	NOT NULL, UNIQUE	—	—
Income_Categories	description	VARCHAR(200)	—	—	—

Table 5.2.5 Relational Schema for Expenses

Table	Attributes	Type	Constraints	PK/FK	FK References
Expenses	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	expense_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	expense_category	VARCHAR(50)	NOT NULL	—	—
	amount	FLOAT	NOT NULL	—	—
	description	VARCHAR(200)	NOT NULL	—	—
	date	DATE	NOT NULL	—	—

Table 5.2.6 Relational Schema for Expense_Categories

Table	Attributes	Type	Constraints	PK/FK	FK References
Expense_Categories	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
Expense_Categories	name	VARCHAR(100)	NOT NULL, UNIQUE	—	—
Expense_Categories	description	VARCHAR(200)	—	—	—

Table 5.2.7 Relational Schema for Employees

Table	Attributes	Type	Constraints	PK/FK	FK References
Employees	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	employee_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	first_name	VARCHAR(100)	NOT NULL	—	—
	last_name	VARCHAR(100)	NOT NULL	—	—
	email	VARCHAR(100)	NOT NULL, UNIQUE	—	—
	phone	VARCHAR(20)	—	—	—
	role	VARCHAR(100)	NOT NULL	—	—
	salary	FLOAT	NOT NULL	—	—
	status	VARCHAR(20)	DEFAULT 'active'	—	—
	profile_image	VARCHAR(255)	—	—	—
	address	TEXT	—	—	—

Table 5.2.8 Relational Schema for Material

Table	Attributes	Type	Constraints	PK/FK	FK References
Materials	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	name	VARCHAR(100)	NOT NULL, UNIQUE	—	—

Table 5.2.9 Relational Schema for Units

Table	Attributes	Type	Constraints	PK/FK	FK References
Units	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	name	VARCHAR(50)	NOT NULL, UNIQUE	—	—

Table 5.2.10 Relational Schema for Sales

Table	Attributes	Type	Constraints	PK/FK	FK References
Sales	id	INT	AUTO_INCREMENT, NOT NULL	PK	–
	sale_id	VARCHAR(50)	NOT NULL, UNIQUE	–	–
	customer_id	INT	NOT NULL	FK	Customers(id)
	employee_id	INT	NOT NULL	FK	Employees(id)
	sale_amount	FLOAT	NOT NULL	–	–
	sale_date	DATE	NOT NULL	–	–
	sale_address	VARCHAR(200)	–	–	–
	sale_paid_amount	FLOAT	NOT NULL	–	–
	sale_unpaid_amount	FLOAT	NOT NULL	–	–
	sale_status	VARCHAR(50)	NOT NULL	–	–

Table 5.2.11 Relational Schema for Purchases

Table	Attributes	Type	Constraints	PK/FK	FK References
Purchases	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	purchase_id	VARCHAR(50)	NOT NULL, UNIQUE	—	—
	purchase_date	DATE	NOT NULL	—	—
	supplier	VARCHAR(50)	NOT NULL	—	—
	material	VARCHAR(50)	NOT NULL	—	—
	unit	VARCHAR(50)	NOT NULL	—	—
	quantity	INT	NOT NULL	—	—
	amount	FLOAT	NOT NULL	—	—

Table 5.2.12 Relational Schema for Reports

Table	Attributes	Type	Constraints	PK/FK	FK References
Reports	id	INT	AUTO_INCREMENT, NOT NULL	PK	—
	report_id	VARCHAR(20)	NOT NULL, UNIQUE	—	—
	report_name	VARCHAR(255)	NOT NULL	—	—
	total_sales	DECIMAL(15,2)	NOT NULL	—	—
	total_purchases	DECIMAL(15,2)	NOT NULL	—	—
	total_income	DECIMAL(15,2)	NOT NULL	—	—
	total_expenses	DECIMAL(15,2)	NOT NULL	—	—
	profit	DECIMAL(15,2)	NOT NULL	—	—
	start_date	DATETIME	NOT NULL	—	—
	end_date	DATETIME	NOT NULL	—	—
	report_period	VARCHAR(20)	NOT NULL	—	—
	notes	TEXT	DEFAULT "	—	—
	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP, NOT NULL	—	—
	updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	—	—

5.3 CRUD Matrix

Table 5.3.1 CRUD Matrix for Admin

Entity/Feature	Manage Users	Manage Customer	Manage Employee	Manage Sales	Manage Suppliers	Manage Purchases	Manage Income	Manage Expenses
Users	C,R,U,D	R	-	-	-	-	-	-
Customers	-	C,R,U,D	-	R	-	-	-	-
Employees	-	-	C,R,U,D	R	-	R	-	-
Sales	-	-	-	C,R,U,D	-	-	R	-
Suppliers	-	-	-	-	C,R,U,D	R	-	R
Purchases	-	-	-	-	-	C,R,U,D	-	R
Income	-	-	-	-	-	-	C,R,U,D	-
Expenses	-	-	-	-	-	-	-	C,R,U,D
Products	-	-	-	R	-	R	R	-

Table 5.3.3 CRUD Matrix for Finance Officer

Entity/Feature	Forecast Finance	Compare Budget	View Reports
Finance	C	R	-
Income	R	R	-
Expenses	R	R	-
Report	R	-	R

Table 5.3.4 CRUD Matrix for Staff

Entity/Feature	View Customers	Add Sales	View Sales
Customer	R	R	R
Sale	-	C	-
Employee	-	R	R
Products	-	R	R

6. Application Development

6.1 Tools & Technologies Used

The application is developed using a combination of modern and reliable technologies to ensure efficiency, scalability, and maintainability.

- **Database Management System (DBMS):** The system uses **MySQL** (via **XAMPP**) as the database management system. MySQL provides efficient relational data storage, retrieval, and management. It supports structured queries, data integrity, indexing, and transaction management, ensuring reliable and secure handling of user and system data.
- **Backend Development:** The backend is developed using **Python with the Flask framework**. Flask provides a lightweight and flexible structure for building web applications, handling server-side logic, routing, API development, and database interactions. Its simplicity allows for easy integration with other tools and libraries.
- **Frontend Development:** The frontend is developed using **JavaScript**, which enhances interactivity and responsiveness. It manages client-side behavior, form validation, dynamic content updates, and user interface interactions, providing a seamless experience.

These technologies are chosen to balance ease of development, performance, and scalability, making the system efficient and maintainable.

6.2 User Interface Design

Login form: Admin can login to gain access to the system.

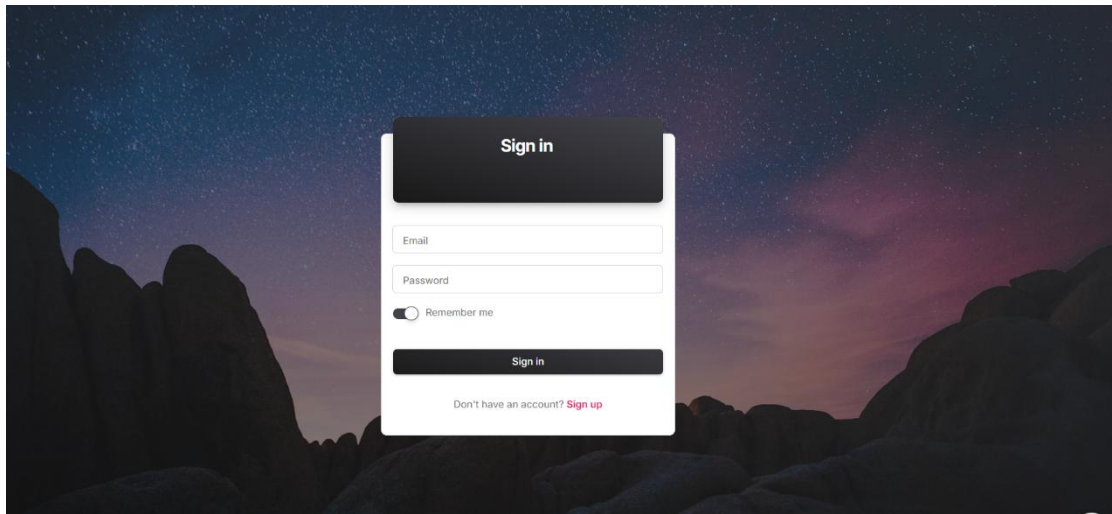


Fig. 6.2.1 Admin Log In Page

Dashboard: This is a business dashboard displaying key performance metrics. It shows current sales, user data, and ad views, alongside historical trends for website traffic, daily sales, and completed tasks.

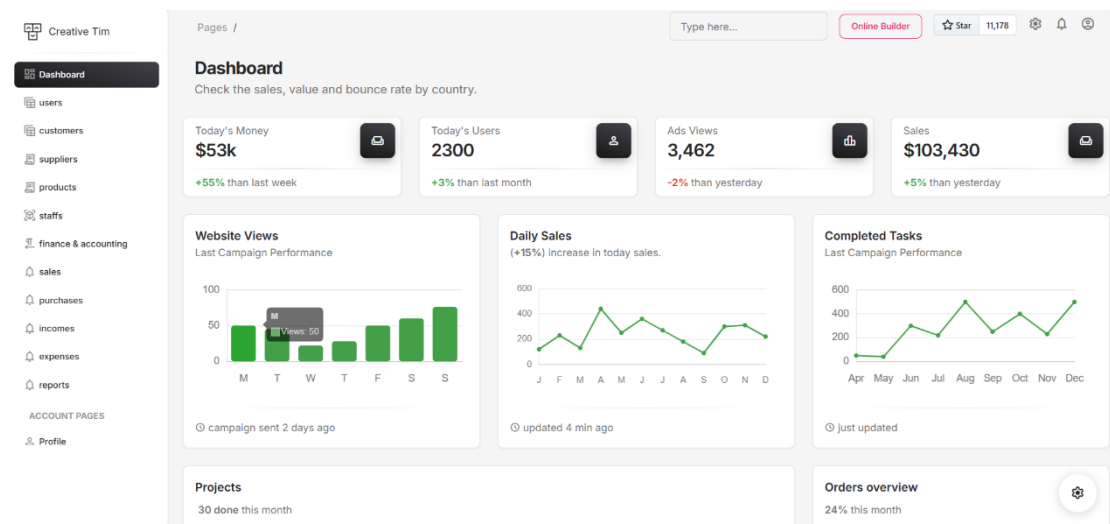


Fig. 6.2.2 Dashboard

Budget and Finance Forecasting (1): Budget and Finance Forecasting: This page is a financial forecasting tool. It allows users to set up a budget by inputting projected revenue streams and expenses for specific departments, projects, and time periods.

Budget & Finance Forecasting

Time Period: Monthly | Department / Project: Marketing | Currency: MMK (Ks)

Revenue Streams + Add Revenue

Sale: 300000 | Monthly | 10

Expenses + Add Expense

Variable: Labor Cost | 30000 | Monthly

Forecasting Assumptions

Fig. 6.2.3 Budget and Finance Forecasting

Budget and Finance Forecasting (2): This figure demonstrates the output of the system's integrated Cash Flow and Financial Forecasting module. The module automatically consolidates data from sales, operations, and procurement to generate a detailed 12-month projection.



Fig. 6.2.4 Budget and Finance Forecasting

Income Management: This screenshot depicts the sales order creation screen within the system. It captures the process of adding a new sales order for a customer, including product selection with quantity and price, automatic calculation of item amounts and the total order value, and the entry of key details such as sale date and delivery method.

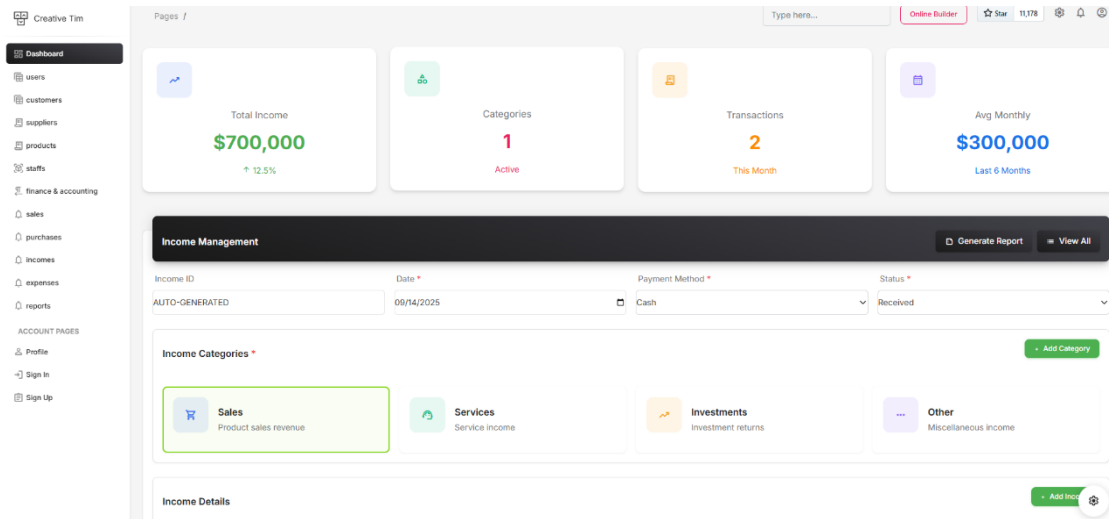


Fig. 6.2.5 Income Management

Sale: This screenshot depicts the sales order creation screen within the system. It captures the process of adding a new sales order for a customer, including product selection with quantity and price, automatic calculation of item amounts and the total order value, and the entry of key details such as sale date and delivery method.

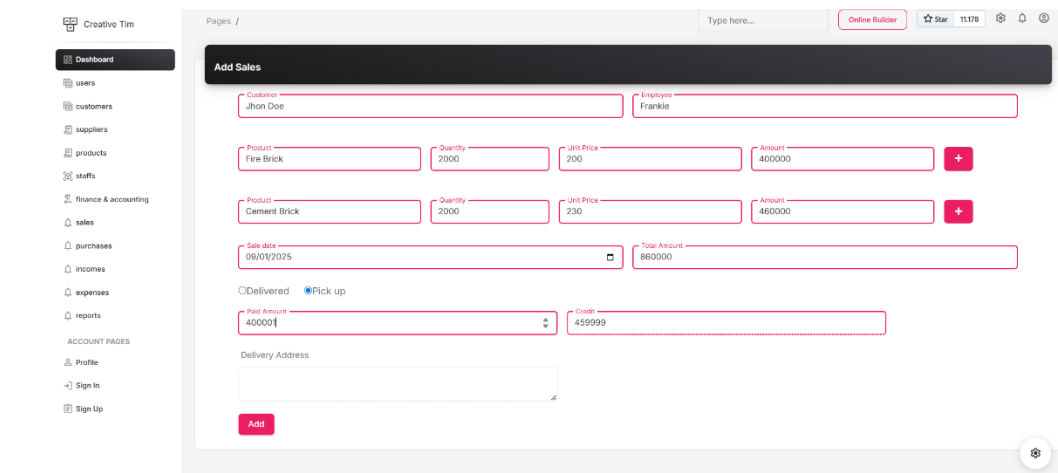


Fig. 6.2.6 Sale

Report: The system's financial dashboard, displaying key metrics, profit trends, and a summary of sales, purchases, income, expenses, and net profit.

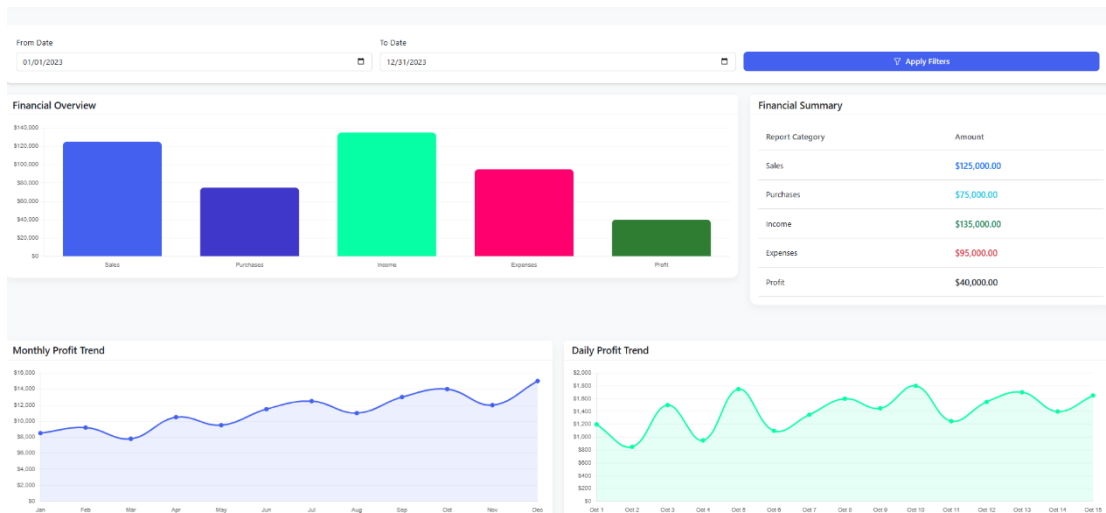


Fig. 6.2.7 Monthly and Daily reports

7. Implementation Details

7.1 How the database is connected to the application

The database is connected via the **Data Access Layer (DAL)** using the **SQLAlchemy ORM**. This connection setup is crucial for advanced database management:

- **Connection Pooling:** The ORM framework is configured to use connection pooling, allowing the application to reuse established database connections instead of opening and closing a new connection for every request. This drastically reduces connection overhead, which is a major factor in database performance and scalability under high load.
- **Secure Parameters:** Database credentials and connection parameters (host, port, database name) are managed securely outside the core application code, typically via environment variables, and are used by the ORM to establish the physical connection.

7.2 Key Modules or Features

1. User Authentication and Authorization

- Secure login system for different roles (Admin, Manager, Finance Officer, Staff).
- Role-based access control to restrict or allow CRUD operations.

2. Customer Management

- Add, edit, view, and delete customer records.
- Store customer details such as name, email, phone, and address.

3. Employee Management

- Maintain employee records (personal details, department, salary, manager).
- Update status, assign roles, and manage employee hierarchy.

4. Product Management

- CRUD operations on products (name, category, price, quantity).
- Track available stock and categories.

5. Sales Management

- Record sales transactions with linked customer and employee data.
- Track paid/unpaid amounts and sales status.

6. Purchases Management

- Maintain records of raw material purchases, suppliers, and costs.
- Link with units and quantities for better tracking.

7. Income & Expense Management

- Categorize and track income sources and expenses.
- Generate summaries for financial monitoring.

8. Report Generation

- Automatic reports for total sales, purchases, income, expenses, and profit.
- Period-based report creation with export capability.

9. Dashboard & Analytics

- Overview of financial health with charts and key figures.
- Highlight trends for sales, expenses, and profits.

8. Testing and Validation

Table 14: Samples test case and expected vs actual results for Business Management System

Test CaseID	Module Tested	Description	Precondition	Test steps	Expected Result	Actual Result	Status
TC-01	User Management	Add a new user	Admin/Manager logged in	1. Go to Manage Users → Add 2. Enter valid user details 3. Save	New user is saved and listed	Works as expected.	Passed
TC-02	Customer Management	Add a new customer	Admin/Manager/Staff logged in	1. Go to Manage Customers → Add 2. Enter details 3. Save	Customer record is created	Works as expected.	Passed
TC-03	Sale Transaction	Record a new sale with products	Staff logged in; products exist	1. Go to Add Sale 2. Add products 3. Confirm sale	Sale transaction recorded	Works as expected.	Passed
TC-04	Purchase Transaction	Record a new purchase	Manager logged in	1. Go to Add Purchase 2. Enter details + Add Materials 3. Save	Purchase transaction recorded	Works as expected.	Passed
TC-05	Reports & Filters	View report with filter	Data exists (sales, purchase, income, expense)	1. Go to Reports 2. Select report type 3. Apply filter	Filtered report displayed	Works as expected.	Passed
TC-06	Expense Management	Add expense record	Expense category exists	1. Go to Add Expense 2. Enter details 3. Save	Expense recorded successfully	Works as expected.	Passed
TC-07	Supplier Management	Add a new supplier	Admin/Manager/Staff logged in	1. Go to Manage Suppliers → Add 2. Enter supplier details 3. Save	Supplier record is created	Works as expected.	Passed
TC-08	Product Management	Add a new product	Admin/Manager/Staff logged in	1. Go to Manage Products → Add 2. Enter product details 3. Save	Product added successfully	Works as expected.	Passed

9. Results and Discussion

9.1 How the System Met the Objectives

The developed system successfully achieved the objectives outlined in the earlier sections. Key points include:

- **Efficient Data Management:** The system allows users to easily add, edit, and manage records, significantly reducing manual effort compared to paper-based or ad-hoc systems. Data is stored securely in the MySQL database, and all operations are reflected in real-time.
- **User-Friendly Interface:** Through the intuitive web-based interface developed using Flask and JavaScript, users can navigate the system without extensive training. Forms, dashboards, and menus are clearly organized, improving accessibility and efficiency.
- **Security and Access Control:** Role-based authentication ensures that only authorized users can access sensitive data or perform critical operations. Passwords are securely hashed, and session management prevents unauthorized access.
- **Reporting and Analytics:** The system generates reports and visualizes data in tables and charts, providing meaningful insights for decision-making. Users can quickly view summaries or analyze trends without manual calculations.
- **System Reliability:** By using Flask for the backend and MySQL for data storage, the system is stable and capable of handling multiple simultaneous users without data loss or errors.

Overall, the system meets the functional and non-functional requirements effectively, addressing the key problems identified in the problem statement.

9.2 Performance and Usability Feedback

During testing and initial usage, the following performance and usability observations were noted:

- **Performance:**
 - Database queries execute efficiently, even with multiple concurrent users.
 - Page load times are minimal, ensuring a smooth user experience.
 - The system handles CRUD operations quickly and reliably, with no noticeable delays.
- **Usability Feedback:**
 - Users found the interface intuitive, with clear navigation menus and form layouts.
 - Visual feedback, such as success or error messages, improves user confidence during operations.
 - Reports and dashboards are easy to interpret, making data analysis faster and more accurate.
- **Areas for Improvement:**
 - Additional features like search filters, sorting, and export options for reports could further enhance usability.
 - Mobile responsiveness could be improved to support users accessing the system from different devices.

In conclusion, the system performs reliably, meets its intended objectives, and provides a positive user experience. Feedback collected during testing highlights the system's effectiveness while identifying areas for future enhancement.

10. Conclusion

This project successfully demonstrates the design and development of a system tailored for manufacturing operations. By integrating key modules such as inventory, sales, finance, human resources, and production management into a unified platform, the system enhances efficiency, accuracy, and collaboration across departments. The functional and non-functional requirements addressed in this project ensure that the system is scalable, secure, and user-friendly, meeting the needs of a dynamic manufacturing environment.

Through use case modeling, domain modeling, sequence diagrams, and class diagrams, the project highlights a structured approach to system design, ensuring clarity and maintainability. The implementation further validates the feasibility of the proposed solution, showing how business processes can be streamlined to reduce redundancy and improve decision-making.

In conclusion, this system lays a strong foundation for modern manufacturing operations by providing real-time insights, optimizing resource utilization, and supporting strategic planning. It not only meets current requirements but also offers flexibility to adapt as organizations grow and face new challenges in the future.

11. References

- <https://stackoverflow.com/questions/7478366/create-dynamic-urls-in-flask-with-url-for>
- [geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/](https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/)
- <https://jquery.com/>
- <https://steven-giesel.com/blogPost/9fa7fe83-3ede-4ecb-ab27-4012b1333c0e>